

Uncovering IPL's Hidden Match- Winners

A Data-Driven SQL Case Study

Project: IPL SQL case study using data from the 2018 and 2019 seasons.

Moving Beyond Basic Averages

The Analytical Goal

Cricket franchises spend millions at the auction table, but finding true, consistent value across multiple seasons requires deeper digging.

- ▶ Who are the consistent run accumulators across both years?
- ▶ Which bowlers provide maximum impact through high wickets or low economy rates?
- ▶ How did team dominance shift between 2018 and 2019?

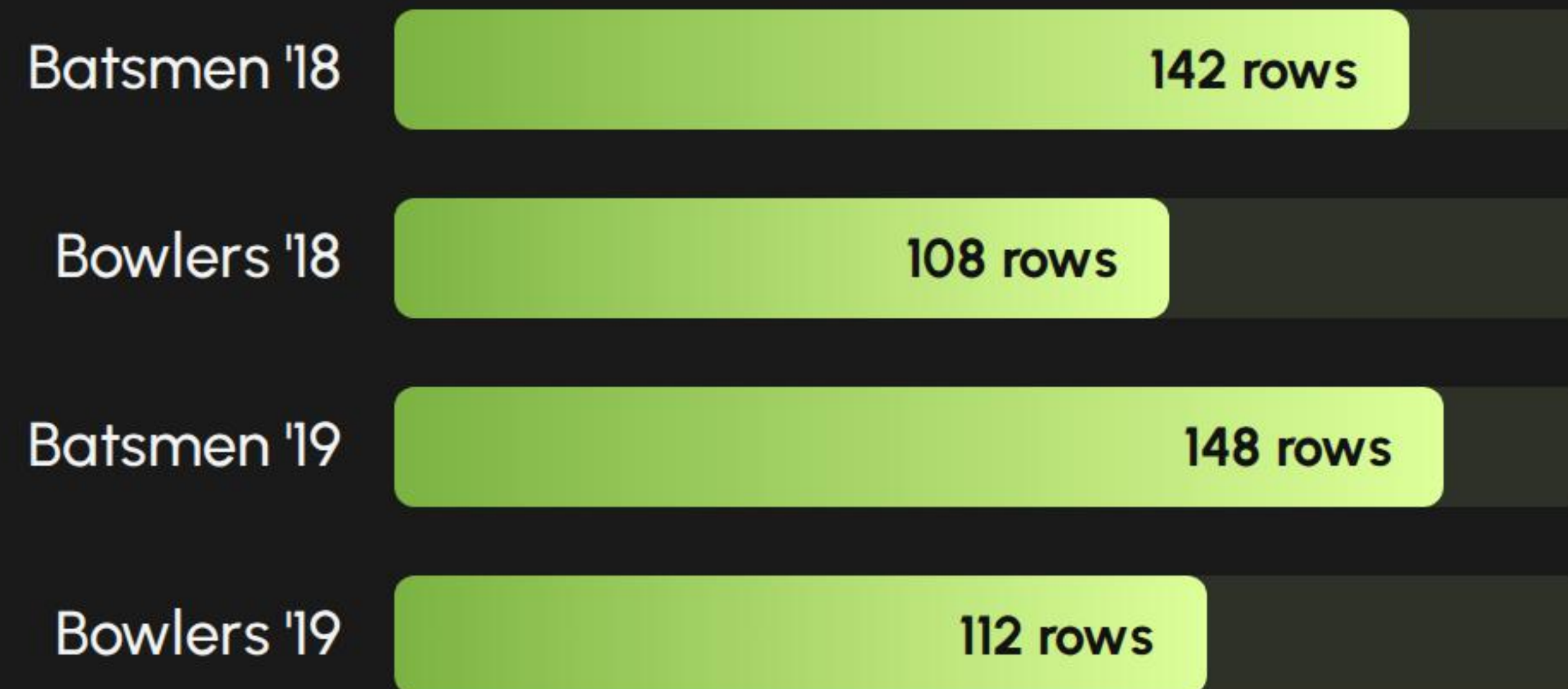
By writing advanced SQL queries against historical player stats, I aimed to extract actionable insights on player efficiency and team performance.

Recruiter Takeaway:

Translates ambiguous business questions (e.g., "Who are the best players?") into structured SQL queries to extract objective answers.

The Data Engine: Merging Two Seasons

Dataset Records by Table



Volume of player records processed across schemas

Dataset Schema & Workflow

The dataset comprises four primary tables splitting batsmen and bowlers across the 2018 and 2019 seasons.

- **Tables:** batsmen_2018, bowlers_2018, batsmen_2019, bowlers_2019
- **Metrics:** Matches, Innings, Runs, Strike Rate, Economy, Wickets
- **Workflow:** Created database -> Imported CSV data -> Executed exploratory & aggregate queries.

💡 **SQL Skill:** DDL (Data Definition Language) & Database Creation.

One team dominated consistency in 2019

Rank	Team	Total Runs (2019)
1	[Team 1]	[Max Runs]
2	[Team 2]	[Runs]
3	[Team 3]	[Runs]
4	[Team 4]	[Runs]
5	[Team 5]	[Runs]

Aggregating Team Power

Analyzing total runs aggregated by team in 2019 reveals a stark gap between the top offensive unit and the rest of the league.

[Insert Screenshot: Output of Query 8 (SUM runs GROUP BY Team)]

 **SQL Skill:** GROUP BY & Aggregate Functions (SUM).

 **Recruiter Takeaway:** Can aggregate granular row-level data into high-level organizational insights.

The toss advantage was weaker than expected



The Myth

Conventional wisdom states that winning the toss offers a significant advantage in T20 cricket, allowing teams to dictate the match conditions.



The Reality

Data indicates a marginal win-rate delta for teams winning the toss. [Team X] actually performed better across 2018-2019 when losing the toss and adapting.

💡 **SQL Skill:** Conditional Aggregation (CASE WHEN) & Percentage Calculations.

🎯 **Recruiter Takeaway:** Uses data to challenge business assumptions rather than just pulling numbers.

Consistency is rare: Kings of runs (2018-19)

To find truly elite performers, we must look across multiple seasons. By stacking the 2018 and 2019 datasets, we identified the most reliable run-scorers over a 2-year period.

[Insert Screenshot: Query 4 - UNION ALL & GROUP BY]

Rank	Player	Total Runs (2 Seasons)
1	[Player Name A]	[Total Runs]
2	[Player Name B]	[Total Runs]
3	[Player Name C]	[Total Runs]
4	[Player Name D]	[Total Runs]
5	[Player Name E]	[Total Runs]

- 💡 **SQL Skill:** UNION ALL & Derived Tables.
- 🎯 **Recruiter Takeaway:** Ability to stitch fragmented datasets vertically for longitudinal analysis.

Chasing told a **very different story** in 2019

65%

Matches Won
Chasing

35%

Matches Won
Defending

The Chase Bias

In 2019, teams batting second had a massive statistical advantage. This insight is crucial for team strategy and predictive modeling.

[Insert Screenshot: SQL query isolating win results by batting order]

💡 **SQL Skill:** Filtering & Ratio Analysis.

🎯 **Recruiter Takeaway:** Uncovers macro-trends that inform high-level business/sports strategy.

Efficiency over volume: **Strike Rates**

Elite Strike Rates (S/R > 130)



Top 4 efficient scorers filtered from Query 2

Total runs are a vanity metric. True impact in T20 cricket comes from scoring fast and maximizing deliveries faced.

≥ 130

Elite Strike Rate
Filter

< 7.0

Elite Economy
Filter

💡 **SQL Skill:** Complex WHERE clauses and Operators (>, <, >=).

🎯 **Recruiter Takeaway:** Understands quality metrics over vanity metrics.

SQL Toolkit Demonstrated

Data Aggregation



Used `'GROUP BY'` and aggregate functions (`'SUM'`, `'MAX'`) to distill thousands of rows into team and player summaries.

Dataset Stitching



Leveraged `'UNION ALL'` to stack yearly data vertically, and `'INNER JOIN'` to combine batting and bowling metrics horizontally.

Advanced Filtering



Applied multi-condition `'WHERE'` and `'HAVING'` clauses to isolate elite performers based on complex criteria (e.g., `Economy < 7`).

Subqueries & Derived Tables



Utilized inline views / subqueries to create temporary combined datasets before performing final aggregations and limits.